GKE Lab: Standard Zonal Cluster and Super Mario Deployment

This guide creates a Standard GKE cluster in a single zone, deploys the **docker.io/pengbai/docker-supermario** container image as a Kubernetes Deployment, exposes it with a Service of type LoadBalancer, and opens the game from a public IP address in a browser.

Prerequisites

- A Google Cloud project with billing enabled.
- Cloud Shell, or a local machine with the gcloud CLI and kubectl installed.
- The IAM role Kubernetes Engine Admin (roles/container.admin) or Owner on the project.

Lab values

Setting	Value used in this guide
Cluster name	mario-cluster
Cluster mode	Standard
Location type	Zonal
Zone	asia-south1-a (Mumbai)
Nodes	2 x e2-medium
Container image	docker.io/pengbai/docker-supermario
Container port	8080
Public port	80

Step 1: Set the project and enable the GKE API

Open Cloud Shell from the Google Cloud console and run the commands below. Replace YOUR_PROJECT_ID with your project ID.

```
gcloud config set project YOUR_PROJECT_ID
gcloud config set compute/zone asia-south1-a
gcloud services enable container.googleapis.com
```

Step 2: Create a Standard zonal cluster

Option A: Using the gcloud CLI

Passing --zone (instead of --region) is what makes the cluster zonal: the control plane and nodes run in one zone.

```
gcloud container clusters create mario-cluster \
  --zone=asia-south1-a \
  --num-nodes=2 \
  --machine-type=e2-medium \
```

```
  --disk-size=30 \
  --release-channel=regular
```

Cluster creation takes about 5 to 10 minutes. When it finishes, confirm the cluster is running:
`gcloud container clusters list`

Option B: Using the Google Cloud console

1. Go to Kubernetes Engine, then Clusters, and click Create.
2. If the console offers Autopilot first, choose Switch to Standard cluster (or Configure under Standard).
3. Under Cluster basics, enter the name mario-cluster.
4. Under Location type, select Zonal and choose the zone asia-south1-a.
5. Under Node pools, open default-pool, set the number of nodes to 2, and under Nodes set the machine type to e2-medium.
6. Leave the other settings at their defaults and click Create.

Step 3: Connect kubectl to the cluster

```
gcloud container clusters get-credentials mario-cluster
--zone=asia-south1-a
kubectl get nodes
```
You should see two nodes with STATUS Ready.

Step 4: Create the Deployment

Create a file named mario-deployment.yaml:
```
cat <<EOF > mario-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mario
  labels:
    app: mario
spec:
  replicas: 2
  selector:
    matchLabels:
      app: mario
  template:
    metadata:
      labels:
        app: mario
    spec:
      containers:
      - name: mario
        image: docker.io/pengbai/docker-supermario:latest
```

```
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: 100m
            memory: 256Mi
          limits:
            memory: 512Mi
EOF
```

Apply it and wait for the Pods to start. The image is about 320 MB, so the first pull can take a minute or two.

```
kubectl apply -f mario-deployment.yaml
kubectl rollout status deployment/mario
kubectl get pods -l app=mario -o wide
```

Both Pods should show STATUS Running and READY 1/1.

Step 5: Expose the Deployment with a LoadBalancer Service

The Service listens on port 80 and forwards traffic to port 8080 in the container. Create a file named mario-service.yaml:

```
cat <<EOF > mario-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: mario-svc
spec:
  type: LoadBalancer
  selector:
    app: mario
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
EOF

kubectl apply -f mario-service.yaml
```

GKE now provisions a Google Cloud external passthrough Network Load Balancer, reserves an external IP address, and creates the firewall rule that allows traffic to the nodes. You do not need to create a firewall rule yourself.

Shortcut: the same Service can be created with one command instead of the YAML file:

```
kubectl expose deployment mario --name=mario-svc --type=LoadBalancer
--port=80 --target-port=8080
```

Step 6: Access the game from the public internet

Watch the Service until the EXTERNAL-IP column changes from <pending> to an IP address. This usually takes 1 to 3 minutes. Press Ctrl+C to stop watching.

```
kubectl get service mario-svc --watch
```

Example output:

```
NAME        TYPE           CLUSTER-IP    EXTERNAL-IP     PORT(S)
AGE
mario-svc   LoadBalancer   10.8.12.34    34.93.120.45    80:31234/TCP
2m
```

Print just the IP address:

```
kubectl get service mario-svc -o
jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

Open **http://EXTERNAL-IP** in any browser (use http, not https). The Super Mario game loads in the page. Use the arrow keys to move, A to run and S to jump.

You can also test from Cloud Shell:

```
curl -I http://EXTERNAL-IP
```

An HTTP/1.1 200 response confirms the application is reachable from outside the cluster.

Step 7: Optional checks

```
kubectl describe service mario-svc
kubectl get endpoints mario-svc
kubectl scale deployment mario --replicas=3
kubectl get pods -l app=mario
```

The endpoints list shows the Pod IPs on port 8080 that receive traffic from the load balancer. After scaling, a third Pod joins the endpoints automatically.

Troubleshooting

Symptom	Likely cause and fix
EXTERNAL-IP stays <pending> for more than 5 minutes	Run kubectl describe service mario-svc and read the Events section. Common causes are an exhausted external IP quota in the region or missing permissions.
Pods show ImagePullBackOff or ErrImagePull	Check the image name in the Deployment. Docker Hub limits anonymous pulls, so wait and retry, or push the image to Artifact Registry and use that path.
Pods stay Pending	The nodes lack CPU or memory. Run kubectl describe pod POD_NAME and check Events, then add nodes or lower the resource requests.
Browser shows connection refused or timeout	Confirm targetPort is 8080, the Service selector matches app: mario, and kubectl get endpoints mario-svc lists Pod IPs.

Step 8: Clean up

Delete the Service first so that GKE releases the load balancer and external IP, then delete the

cluster to stop all charges.

```
kubectl delete service mario-svc
kubectl delete deployment mario
gcloud container clusters delete mario-cluster --zone=asia-south1-a
--quiet
```